Web-Technologien UE - Assignment 5

Raphael Bleier, WS 2026

Exercise 1 - Callbacks

a) Code erklärt

`task(taskLabel, evlog)` ist eine Konstruktorfunktion. Jeder Task bekommt ein Label ("A" oder "B"), eine Referenz auf das Log-Element und seine drei Buttons. Die werden über die id gesucht, die aus Präfix und Label zusammengesetzt ist (`'start' + 'A'` = startA). `counter` zählt, wie oft der Task gestartet wurde. Am Anfang sind End und Cancel deaktiviert.

Auf den Start-Button kommt ein click-Listener, der `runTask` aufruft. Mitgegeben werden der Task selbst und zwei Callbacks: einer für "fertig" (ruft endTask auf) und einer für "abgebrochen" (ruft cancelTask auf).

`runTask` schreibt "started" mit Datum ins Log, deaktiviert Start und aktiviert End und Cancel. Nur beim ersten Start (counter === 0) werden die click-Listener auf End und Cancel gesetzt, die dann den passenden Callback aufrufen. Würde man das bei jedem Start machen, hätte man nach drei Starts drei Listener und alles stünde dreimal im Log. Die Zuweisung an `task.endListener` bringt übrigens nichts, addEventListener gibt immer undefined zurück.

`endTask` und `cancelTask` setzen die Buttons zurück und schreiben "ended" bzw. "cancelled" ins Log. Ganz unten werden die zwei Tasks A und B erzeugt. Die laufen unabhängig voneinander, man kann also A starten, dann B, dann A beenden usw.

b) Higher Order Functions und Callbacks

Ein Callback ist eine Funktion, die man einer anderen Funktion übergibt, damit die sie später aufruft, z.B. wenn etwas fertig ist. Die Funktion, die den Callback entgegennimmt, ist per Definition eine Higher Order Function. Das sind also zwei Seiten derselben Sache: Callbacks gibt es nur, weil Funktionen in JS First Class sind und an Higher Order Functions übergeben werden können. Hier ist `runTask` die Higher Order Function und `endCallback`/`cancelCallback` sind die Callbacks.

c) Wo sind Callbacks?

Die zwei Arrow Functions `() => { endTask(this) }` und `() => { cancelTask(this) }`, die an runTask übergeben werden. Außerdem ist jeder Event Handler ein Callback: die Arrow Function am Start-Button und die beiden in runTask an End und Cancel. Die übergibt man an addEventListener (auch eine Higher Order Function) und der Browser ruft sie beim Klick auf.

d) Warum Arrow Functions?

Weil im Handler `this` der Task sein muss. Arrow Functions haben kein eigenes this, sie nehmen das this von dort, wo sie definiert sind. Das ist hier der Konstruktor, also das neue Task-Objekt.

Mit normalen `function () {...}` ruft der Browser den Handler mit this = Button auf. Dann wird `runTask(this, ...)` mit dem Button statt dem Task aufgerufen. Der Button hat kein `evlog`, also ist `task.evlog` undefined, und gleich in der ersten Zeile von runTask kommt:

```
Uncaught TypeError: Cannot read properties of undefined (reading 'innerHTML')
```

Hab ich ausprobiert, genau diese Meldung kommt beim Klick auf "Start A". Bei den inneren Callbacks wäre this sogar undefined bzw. window, weil sie als ganz normale Funktion aufgerufen werden.

Exercise 2 - Promises I

a) Der neue Konstruktor

Statt zwei Callbacks zu übergeben, ruft der Start-Listener `runTaskPromise(this)` auf. Das gibt ein Promise zurück, also ein Objekt für ein Ergebnis, das erst später kommt. Mit `.then(...)` sagt man, was passieren soll, wenn das Promise erfüllt (resolved) wird, nämlich endTask. Mit `.catch(...)`, was bei einer Ablehnung (rejected) passiert, nämlich cancelTask. Der Ablauf ist derselbe wie vorher, nur kümmert sich runTaskPromise nicht mehr darum, wer was danach macht. Es meldet nur "fertig" oder "abgebrochen".

b) runTaskPromise

```
return new Promise((resolve, reject) => {
    task.endButton.addEventListener('click', () => resolve(), { once: true });
    task.cancelButton.addEventListener('click', () => reject('cancelled'), { once: true });
});
```

Klick auf End ruft resolve auf und damit läuft das then. Klick auf Cancel ruft reject auf und damit läuft das catch. Mit `once: true` wird ein Listener nach dem ersten Auslösen wieder entfernt. Bei jedem Start entsteht ja ein neues Promise mit neuen Listenern, ohne once würden sich die anhäufen.

c) Unterschied Callbacks und Promises

In beiden Dateien wird nach dem Abbrechen der End-Button wieder aktiviert (`ex2c_callbacks.html` und `ex2c_promises.html`). Dann jeweils: Start, Cancel, End.

Callback-Version: Im Log steht "A started, A cancelled, A ended". Der End-Listener hängt dauerhaft am Button und ruft bei jedem Klick den Callback auf, egal was vorher passiert ist. Ein Task kann also abgebrochen *und* beendet werden, was keinen Sinn macht.

Promise-Version: Im Log steht nur "A started, A cancelled", der Klick auf End macht nichts mehr. Ein Promise kann nur einmal entschieden werden. Ist es einmal rejected, wird ein späteres resolve() einfach ignoriert und das then läuft nie. Dass then und catch nur einmal und nur exklusiv ausgeführt werden, ist der große Vorteil von Promises. Bei Callbacks muss man selbst aufpassen, dass sie nicht mehrmals oder beide aufgerufen werden.

Exercise 3 - Promises II

a) Was passiert im Original?

`delayedCalc(a, b)` gibt ein Promise zurück. Wenn a > b ist, wird nach einer zufälligen Zeit zwischen 0 und 2 Sekunden die Summe ins Log geschrieben und das Promise resolved. Sonst wird es sofort rejected und "rejected" ins Log geschrieben.

Die vier Aufrufe laufen alle gleichzeitig los, keiner wartet auf den anderen, und die Ergebnisse werden auch nicht weitergegeben. 7,14 und 23,25 werden sofort rejected (b ist größer), deshalb steht "rejected" immer zweimal ganz oben. 5+3 und 1000+1 kommen danach, aber in zufälliger Reihenfolge je nachdem welcher Timeout kürzer ist. Darum ändert sich die Ausgabe beim Neuladen. Weil keiner die Rejections abfängt, kommt in der Konsole noch zweimal "Uncaught (in promise) error: b larger than a".

b) Promise-Kette

```
delayedCalc(5, 3)
    .then(result => delayedCalc(result, 7))
    .then(result => delayedCalc(result, 14))
    .then(result => delayedCalc(result, 23))
    .then(result => delayedCalc(result, 25))
    .then(result => delayedCalc(result, 1000))
    .then(result => delayedCalc(result, 1))
    .catch(error => handleError(error));
```

Jedes then gibt das neue Promise zurück, und das nächste then wartet darauf. So wird alles nacheinander mit dem Zwischenergebnis gerechnet. Bei 77 + 1000 ist b größer, das Promise wird rejected, alle restlichen then werden übersprungen und es geht direkt ins catch. Ausgabe wie verlangt: 5 + 3 = 8 bis 52 + 25 = 77, dann "rejected", und handleError zeigt den Fehler im alert.

c) async/await

Siehe `5_3c.js`. `await` wartet, bis das Promise fertig ist, und gibt direkt das Ergebnis zurück. Bei einem reject wirft es eine Exception, die vom try/catch gefangen wird. Gleiche Ausgabe wie b, liest sich aber wie normaler Code von oben nach unten.

Exercise 4 und 5 - XHR und fetch

Beide unter `http://localhost/a5/ex4/products.html` bzw. `ex5` über Apache getestet, die 4 Produkte werden wie auf Blatt 4 als `div.product` gerendert. Die URL ist relativ (`products.json`), dann passt es egal wo der Ordner liegt.

Ex4: `XMLHttpRequest` mit open/send. In onload wird der Status geprüft und dann `JSON.parse`, onerror fängt Netzwerkfehler ab.

Ex5: `fetch(url)` gibt ein Promise zurück. Wichtig bei der Fehlerbehandlung: fetch wird bei einem 404 oder 500 *nicht* rejected, sondern nur bei echten Netzwerkfehlern. Deshalb prüfe ich `response.ok` selbst und werfe einen Error. Das catch am Ende fängt dann beides und zeigt die Meldung rot auf der Seite an. Mit einer falschen URL getestet: "Could not load products (status 404)".

Direkt über das Dateisystem (file://) geht beides nicht, weil der Browser von dort keine Requests erlaubt. Deshalb braucht man den Webserver.